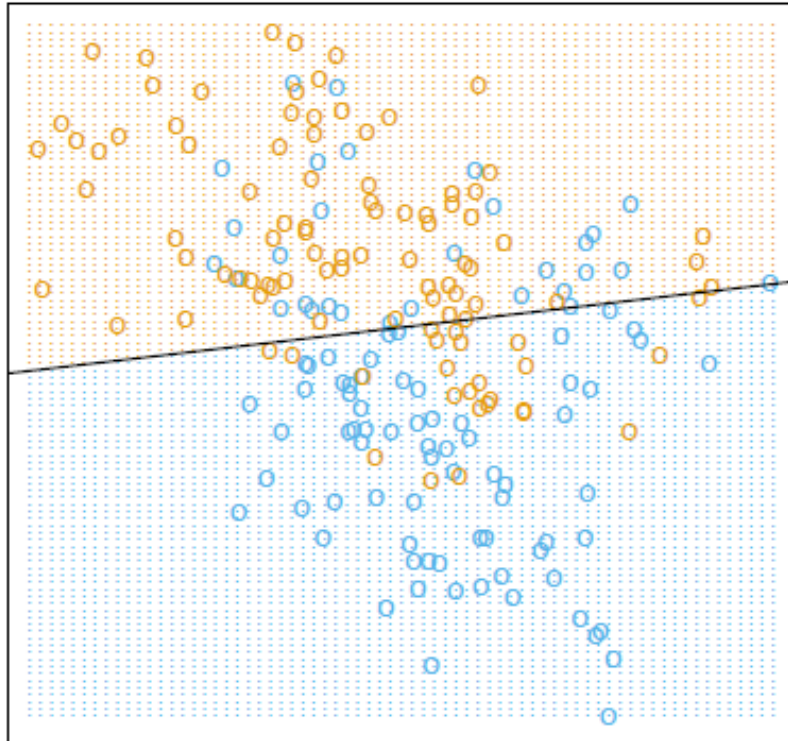


Linear Classification/
Logistic Regression/

Linear models for classification

Linear Regression of 0/1 Response



The classes are coded as a binary variable (BLUE = 0, ORANGE = 1), and then fit by linear regression.

The line is the decision boundary defined by $w^T x = 0.5$.

The orange shaded region denotes that part of input space classified as ORANGE, while the blue region is classified as BLUE.

Classification problems

Given data set $D = \langle x_i, y_i \rangle, i=1:n$, with **discrete** y_i , find a hypothesis which “best fits” the data.

- If $y_i \in \{0, 1\}$ this is **binary** classification.
- If y_i can take more than two values, the problem is called **multi-class** classification.

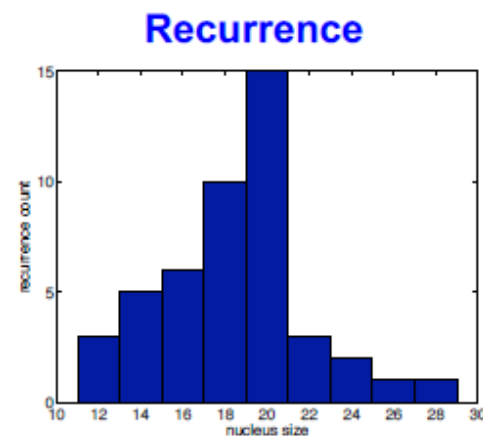
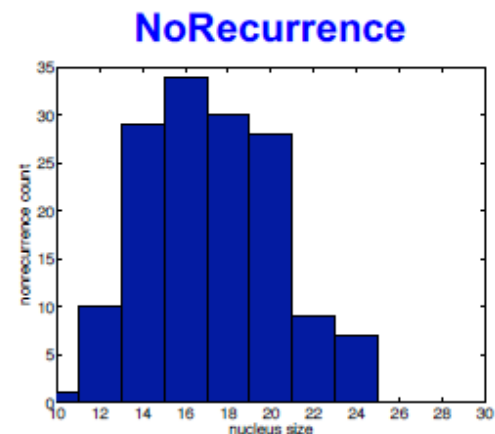
epal.length	sepal.width	petal.length	petal.width	variety
5.1	3.5	1.4	0.2	Setosa
4.9	3	1.4	0.2	Setosa
4.7	3.2	1.3	0.2	Setosa
4.6	3.1	1.5	0.2	Setosa
5	3.6	1.4	0.2	Setosa
5.4	3.9	1.7	0.4	Setosa
4.6	3.4	1.4	0.3	Setosa
5	3.4	1.5	0.2	Setosa
4.6	3.2	1.4	0.2	Setosa
5.3	3.7	1.5	0.2	Setosa
5	3.3	1.4	0.2	Setosa
7	3.2	4.7	1.4	Versicolor
6.4	3.2	4.5	1.5	Versicolor
6.9	3.1	4.9	1.5	Versicolor
5.5	2.3	4	1.3	Versicolor
6.5	2.8	4.6	1.5	Versicolor
5.7	2.8	4.5	1.3	Versicolor
6.3	3.3	4.7	1.6	Versicolor

Applications of classification

- Text classification (spam filtering, news filtering, building web directories, etc.)
- Image classification (face detection, object recognition, etc.)
- Prediction of cancer recurrence.
- Financial forecasting.
- Many, many more!

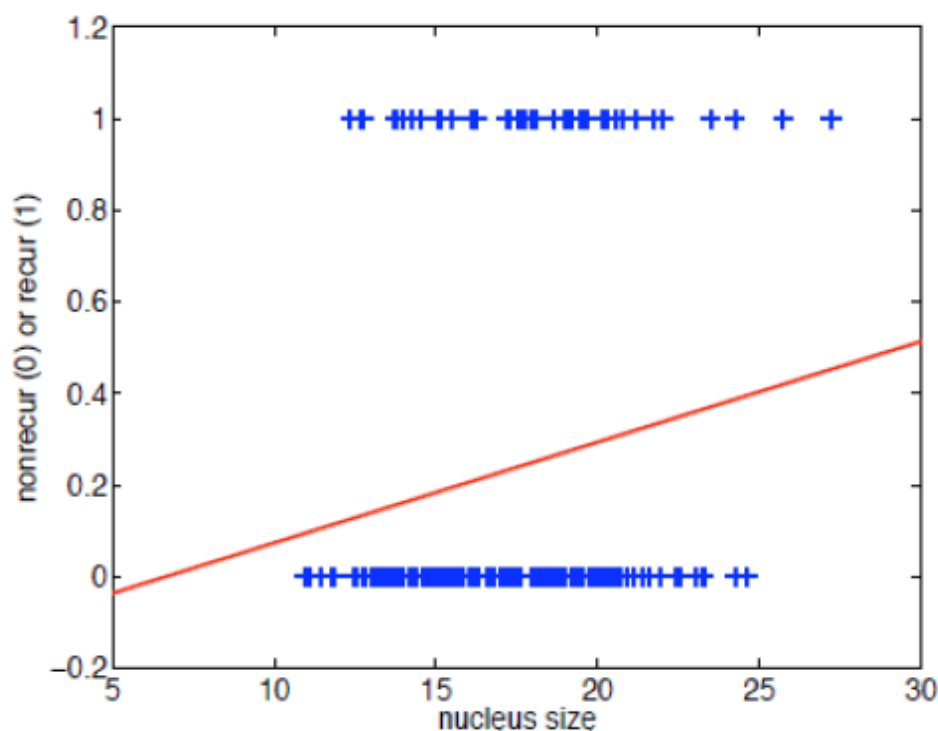
Simple example

- Given “nucleus size”, predict cancer recurrence.
- Univariate input: X = nucleus size.
- Binary output: $Y = \{\text{NoRecurrence} = 0; \text{Recurrence} = 1\}$
- Try: Minimize the least-square error.



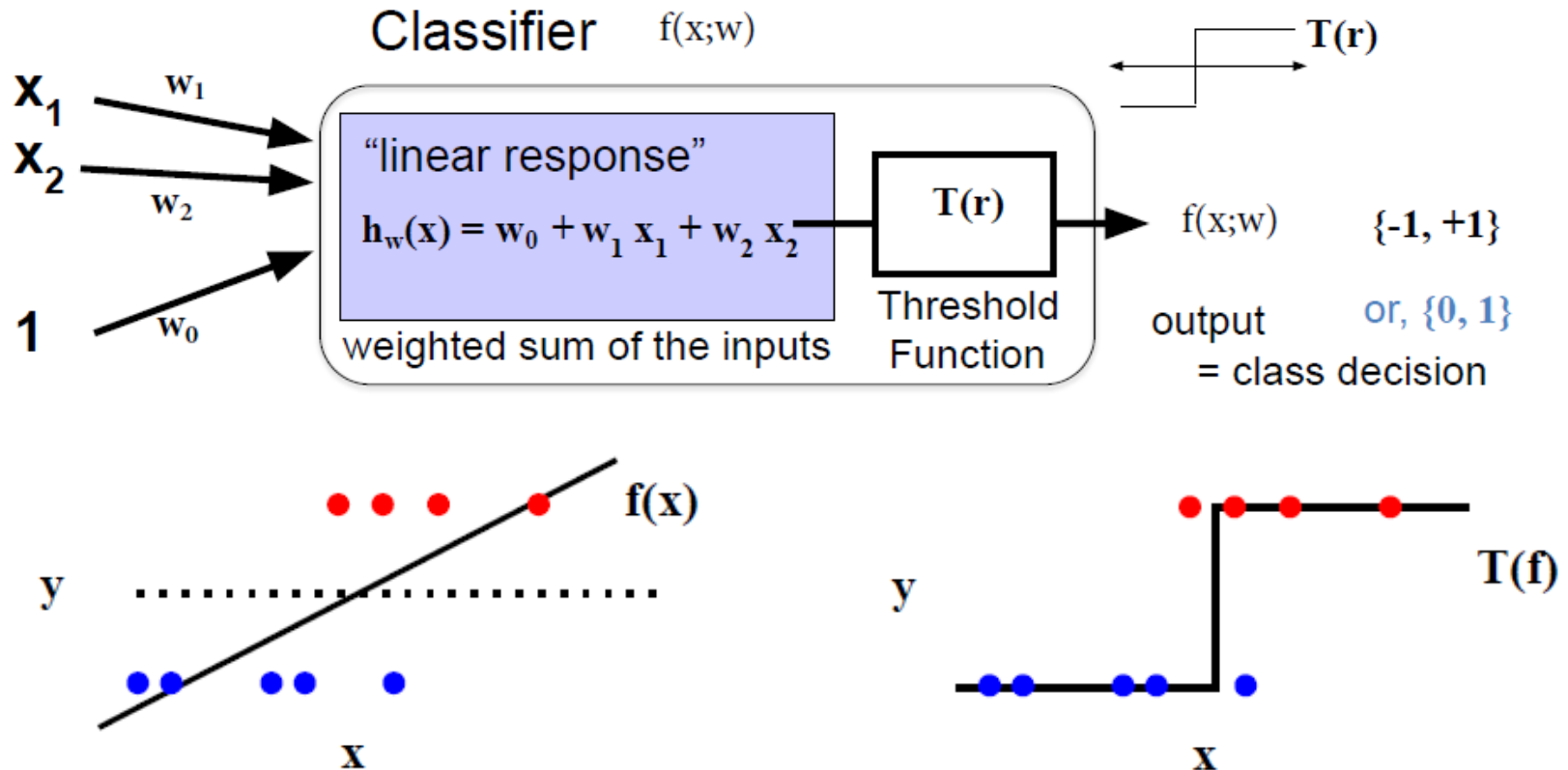
Predicting a class from linear regression

- How to get a binary output?
 - Threshold the output:
 $\{ y \leq t \text{ for NoRecurrence,}$
 $y > t \text{ for Recurrence} \}$
 - Interpret output as probability:
 $y = Pr(\text{Recurrence})$



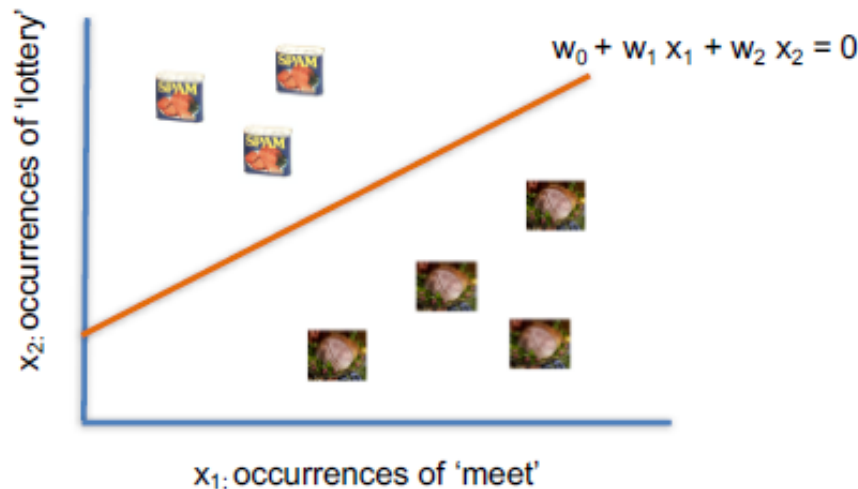
Perceptron

- Perceptron takes weighted linear combination of input features and pass it through a thresholding function which outputs 1 or 0.
- The sign of $w^T x$ tells us which side of the plane $w^T x = 0$, the point x lies on.
- Thus by taking threshold as 0, perceptron classifies data based on which side of the plane the new point lies on.



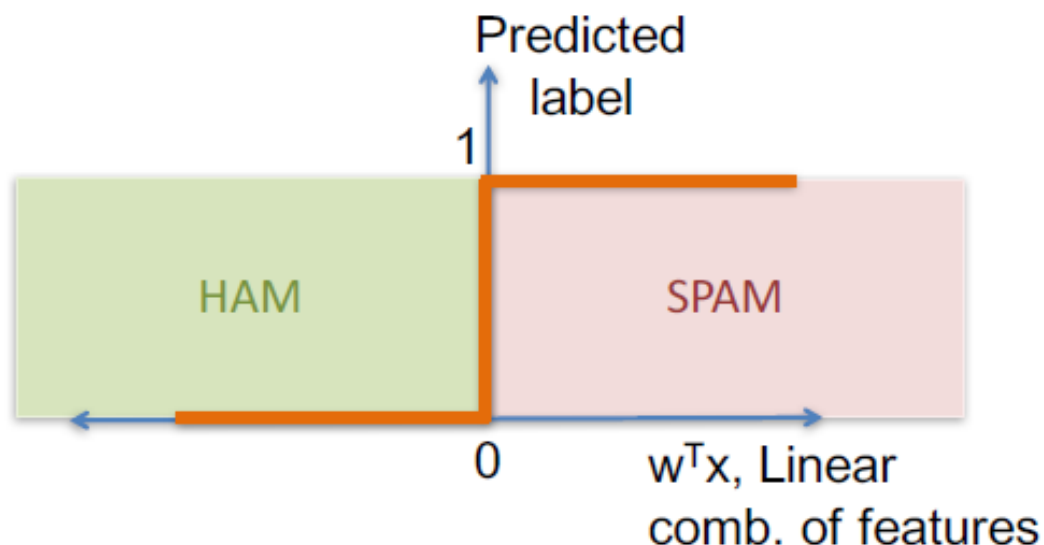
Example

- $x_1 = \#$ of times 'meet' appears in an email
- $x_2 = \#$ of times 'lottery' appears in an email
- Define feature vector $\mathbf{x} = [1, x_1, x_2]$
- Learn the decision boundary $w_0 + w_1 x_1 + w_2 x_2 = 0$ such that
 - If $\mathbf{w}^T \mathbf{x} \geq 0$ declare $y = 1$ (spam)
 - If $\mathbf{w}^T \mathbf{x} < 0$ declare $y = 0$ (ham)



Visualizing a Linear Classifier

- $x_1 = \#$ of times 'lottery' appears in an email
- $x_2 = \#$ of times 'meet' appears in an email
- Define feature vector $\mathbf{x} = [1, x_1, x_2]$
- Learn the decision boundary $w_0 + w_1x_1 + w_2x_2 = 0$ such that
 - If $\mathbf{w}^T \mathbf{x} \geq 0$ declare $y = 1$ (spam)
 - If $\mathbf{w}^T \mathbf{x} < 0$ declare $y = 0$ (ham)



$y = 1$ for spam, $y = 0$ for ham

Suppose you see the following email:

CONGRATULATIONS!! Your email address have won you the lottery sum of US\$2,500,000.00 USD to claim your prize, contact your office agent (Athur walter) via email claims2155@yahoo.com.hk or call +44 704 575 1113

Keywords are [lottery, prize, office, email]

The given weight vector is $\mathbf{w} = [0.3, 0.3, -0.1, -0.04]^T$

Will we predict that the email is spam or ham?

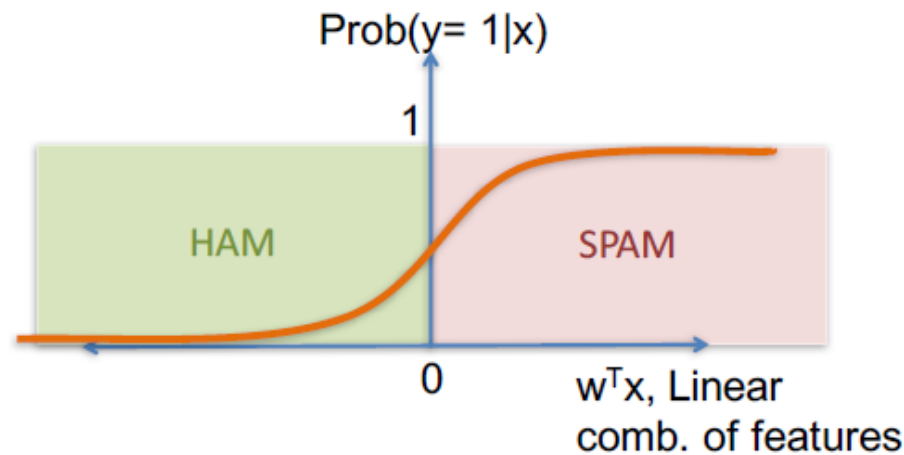
$$\mathbf{x} = [1, 1, 1, 2]^T$$

$$\mathbf{w}^T \mathbf{x} = 0.3 * 1 + 0.3 * 1 - 0.1 * 1 - 0.04 * 2 = 0.42 > 0$$

so we predict spam!

Intuition : Logistic Regression

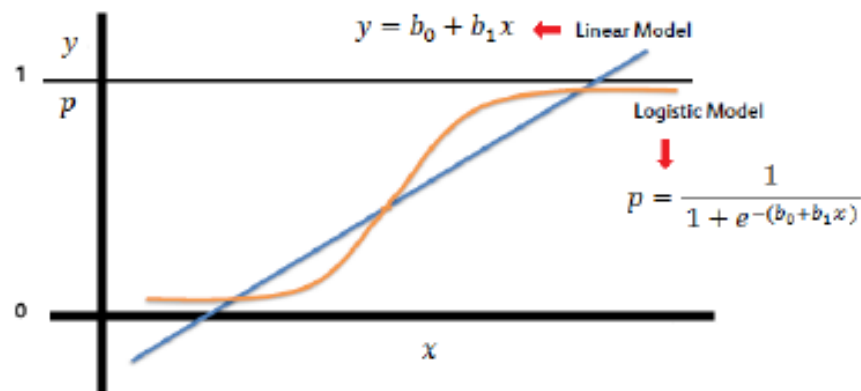
- Suppose we want to output the **probability** of an email being spam/ham instead of just 0 or 1
- This gives information about the confidence in the decision
- Use a function $\sigma(\mathbf{w}^T \mathbf{x})$ that maps $\mathbf{w}^T \mathbf{x}$ to a value between 0 and 1



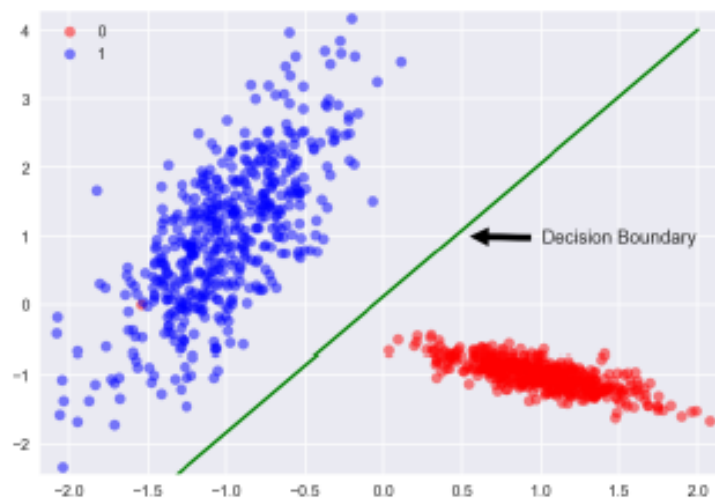
Probability that predicted label is 1 (spam)

Key Problem: Finding optimal weights **w** that accurately predict this probability for a new email

Sigmoid function returns values in $[0,1]$



Decision boundary is linear: linear classifier



Suppose you see the following email:

CONGRATULATIONS!! Your email address have won you the lottery sum of US\$2,500,000.00 USD to claim your prize, contact your office agent (Athur walter) via email claims2155@yahoo.com.hk or call +44 704 575 1113

Keywords are [lottery, prize, office, email]

The given weight vector is $\mathbf{w} = [0.3, 0.3, -0.1, -0.04]^\top$

What is the probability that the email is spam?

$$\mathbf{x} = [1, 1, 1, 2]^\top$$

$$\mathbf{w}^\top \mathbf{x} = 0.3 * 1 + 0.3 * 1 - 0.1 * 1 - 0.04 * 2 = 0.42 > 0$$

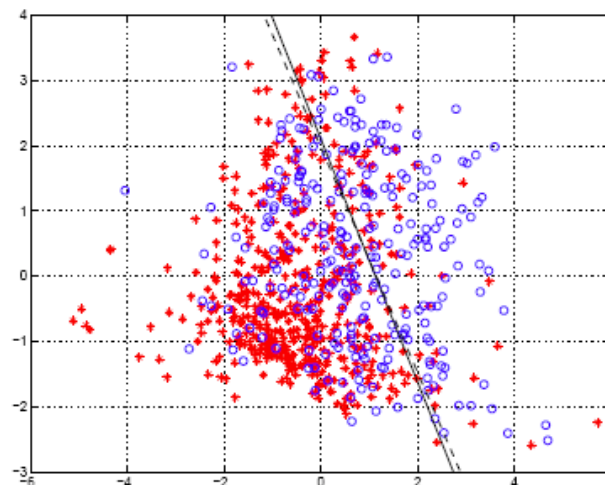
$$\Pr(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-0.42}} = 0.603$$

Classification Based on Probability

- Instead of just predicting the class, give the probability of the instance being that class
 - i.e., learn $p(y \mid \mathbf{x})$
- Recall that:

$$0 \leq p(\text{event}) \leq 1$$

$$p(\text{event}) + p(\neg \text{event}) = 1$$



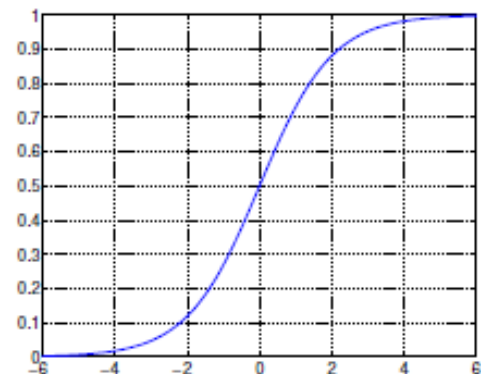
Why the Sigmoid Function?

What does it look like?

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

where

$$a = \mathbf{w}^T \mathbf{x}$$



Sigmoid properties

- Bounded between 0 and 1 ← thus, interpretable as probability
- Monotonically increasing ← thus, usable to derive classification rules
 - $\sigma(a) \geq 0.5$, positive (classify as '1')
 - $\sigma(a) < 0.5$, negative (classify as '0')
- Nice computational properties ← as we will see soon

Interpretation of Hypothesis Output

$$h_w(\mathbf{x}) = \text{estimated } p(y = 1 \mid \mathbf{x}; w)$$

Example: Cancer diagnosis from tumor size

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{tumorSize} \end{bmatrix}$$

$$h_w(\mathbf{x}) = 0.7$$

→ Tell patient that 70% chance of tumor being malignant

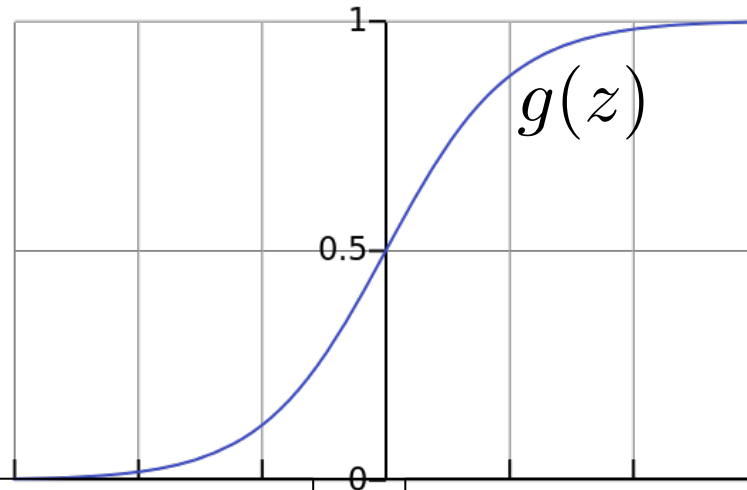
Note that: $p(y = 0 \mid \mathbf{x}; w) + p(y = 1 \mid \mathbf{x}; w) = 1$

Therefore, $p(y = 0 \mid \mathbf{x}; w) = 1 - p(y = 1 \mid \mathbf{x}; w)$

Logistic Regression

$$h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}^T \mathbf{x})$$

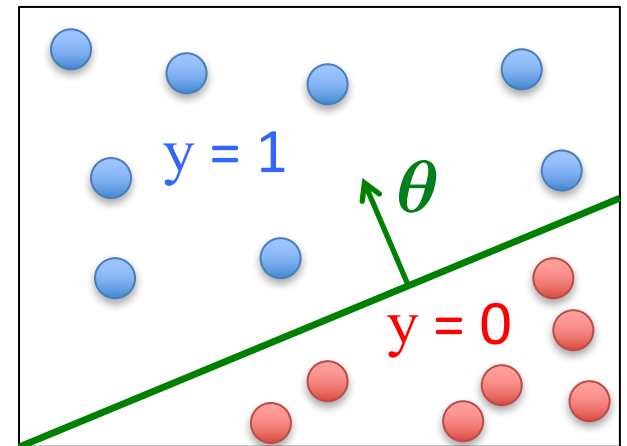
$$g(z) = \frac{1}{1 + e^{-z}}$$



$(\mathbf{w}^T \mathbf{x})$ should be large negative values for negative instances

$(\mathbf{w}^T \mathbf{x})$ should be large positive values for positive instances

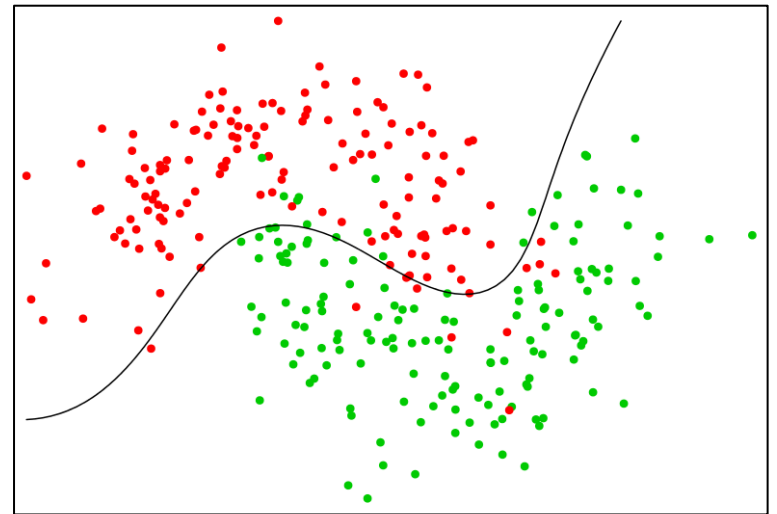
- Assume a threshold and...
 - Predict $y = 1$ if $h_{\mathbf{w}}(\mathbf{x}) \geq 0.5$
 - Predict $y = 0$ if $h_{\mathbf{w}}(\mathbf{x}) < 0.5$



Non-Linear Decision Boundary

- Can apply basis function expansion to features, same as with linear regression

$$\mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_1 x_2 \\ x_1^2 \\ x_2^2 \\ x_1^2 x_2 \\ x_1 x_2^2 \\ \vdots \end{bmatrix}$$



Logistic Regression Objective Function

- Can't just use squared loss as in linear regression:

$$J(\mathbf{w}) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$$

- Using the logistic regression model

$$h_{\mathbf{w}}(\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$$

results in a non-convex optimization

Deriving the Cost Function via Maximum Likelihood Estimation

- Product of the individual likelihoods for each data point when the data points are assumed to be independent.

- Likelihood of data is given by:
$$l(\boldsymbol{\theta}) = \prod_{i=1}^n p(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta})$$

- So, looking for the $\boldsymbol{\theta}$ that maximizes the likelihood

$$\boldsymbol{\theta}_{\text{MLE}} = \arg \max_{\boldsymbol{\theta}} l(\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \prod_{i=1}^n p(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta})$$

- Can take the log without changing the solution:

$$\begin{aligned} \boldsymbol{\theta}_{\text{MLE}} &= \arg \max_{\boldsymbol{\theta}} \log \prod_{i=1}^n p(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}) \\ &= \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^n \log p(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}) \end{aligned}$$

Let us assume that

$$\begin{aligned}P(y = 1 \mid x; \theta) &= h_{\theta}(x) \\P(y = 0 \mid x; \theta) &= 1 - h_{\theta}(x)\end{aligned}$$

Note that this can be written more compactly as

$$p(y \mid x; \theta) = (h_{\theta}(x))^y (1 - h_{\theta}(x))^{1-y}$$

Assuming that the m training examples were generated independently, we can then write down the likelihood of the parameters as

$$\begin{aligned}L(\theta) &= p(\vec{y} \mid X; \theta) \\&= \prod_{i=1}^m p(y^{(i)} \mid x^{(i)}; \theta) \\&= \prod_{i=1}^m (h_{\theta}(x^{(i)}))^{y^{(i)}} (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}}\end{aligned}$$

Deriving the Cost Function via Maximum Likelihood Estimation

- Expand as follows:

$$\begin{aligned} \mathbf{w}_{\text{MLE}} &= \arg \max_{\mathbf{w}} \sum_{i=1}^n \log p(y^{(i)} \mid \mathbf{x}^{(i)}; \mathbf{w}) \\ &= \arg \max_{\mathbf{w}} \sum_{i=1}^n \left[y^{(i)} \log p(y^{(i)} = 1 \mid \mathbf{x}^{(i)}; \mathbf{w}) + (1 - y^{(i)}) \log (1 - p(y^{(i)} = 1 \mid \mathbf{x}^{(i)}; \mathbf{w})) \right] \end{aligned}$$

- Substitute in model, and take negative to yield

Logistic regression objective:

$$\min_{\theta} J(\mathbf{w})$$

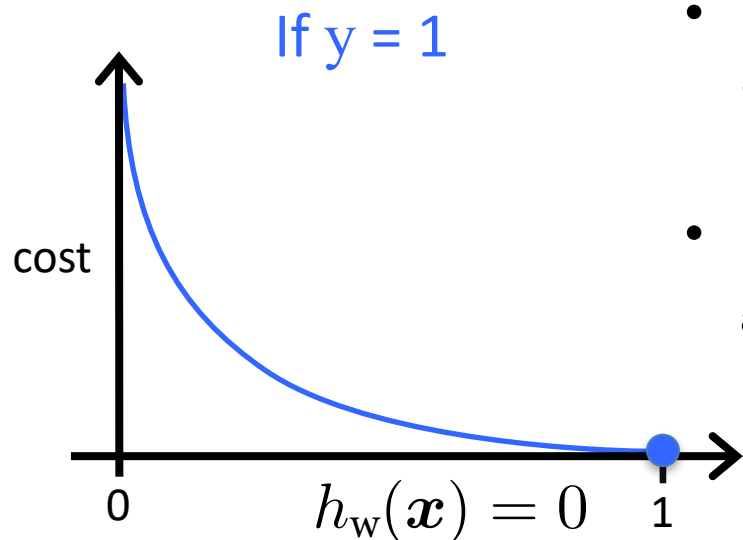
$$J(\mathbf{w}) = - \sum_{i=1}^n \left[y^{(i)} \log h_{\mathbf{w}}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})) \right]$$

Intuition Behind the Objective

$$\text{cost}(h_w(\mathbf{x}), y) = \begin{cases} -\log(h_w(\mathbf{x})) & \text{if } y = 1 \\ -\log(1 - h_w(\mathbf{x})) & \text{if } y = 0 \end{cases}$$

When actual label y is 1

- Cost = 0 if prediction is correct
- As $h_w(\mathbf{x}) \rightarrow 0$, cost $\rightarrow \infty$
- meaning the predicted probability of a positive outcome is very low, the cost for that example increases significantly.
- Captures intuition that larger mistakes should get larger penalties



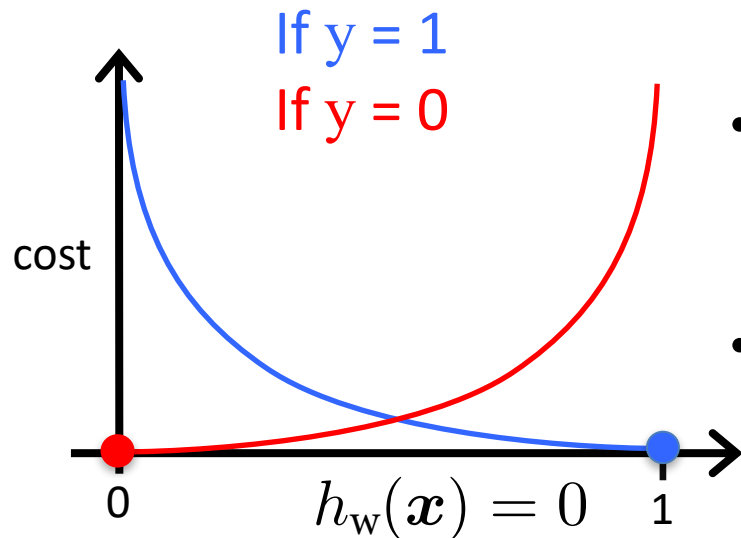
– e.g., predict $h_w(\mathbf{x}) = 0$, but $y = 1$

Intuition Behind the Objective

$$\text{cost}(h_w(\mathbf{x}), y) = \begin{cases} -\log(h_w(\mathbf{x})) & \text{if } y = 1 \\ -\log(1 - h_w(\mathbf{x})) & \text{if } y = 0 \end{cases}$$

If $y = 0$

- Cost = 0 if prediction is correct
- As $(1 - h_w(\mathbf{x})) \rightarrow 0$, $\text{cost} \rightarrow \infty$



- meaning the predicted probability of a negative outcome is very low), the cost for that example increases significantly.
- Captures intuition that larger mistakes should get larger penalties

Gradient Descent for Logistic Regression

$$J_{\text{reg}}(\mathbf{w}) = - \sum_{i=1}^n \left[y^{(i)} \log h_{\mathbf{w}}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})) \right]$$

Want $\min_{\mathbf{w}} J(\mathbf{w})$

- Initialize $\mathbf{w}$
- Repeat until convergence

$$\mathbf{w}_j = \mathbf{w}_j - \alpha \frac{\partial}{\partial \mathbf{w}_j} J(\mathbf{w})$$

simultaneous update
for $j = 0 \dots d$

Use the natural logarithm ($\ln = \log_e$) to cancel with the $\exp()$ in $h_{\mathbf{w}}(\mathbf{x})$

Gradient Descent for Logistic Regression

$$J_{\text{reg}}(\mathbf{w}) = - \sum_{i=1}^n \left[y^{(i)} \log h_{\mathbf{w}}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})) \right]$$

Want $\min_{\mathbf{w}} J(\mathbf{w})$

- Initialize $\mathbf{w}$
- Repeat until convergence (simultaneous update for $j = 0 \dots d$)

$$\mathbf{w}_0 = \mathbf{w}_0 - \alpha \sum_{i=1}^n \left(h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)} \right)$$

$$\mathbf{w}_j = \mathbf{w}_j - \alpha \left[\sum_{i=1}^n \left(h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} \right]$$

Gradient Descent for Logistic Regression

- Initialize $\mathbf{w}$
- Repeat until convergence (simultaneous update for $j = 0 \dots d$)

$$w_0 \leftarrow w_0 - \alpha \sum_{i=1}^n \left(h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)} \right)$$

$$w_j \leftarrow w_j - \alpha \left[\sum_{i=1}^n \left(h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} \right]$$

This looks IDENTICAL to linear regression!!!

- Ignoring the $1/n$ constant
- However, the form of the model is very different:

$$h_{\mathbf{w}}(\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$$

Gradient Descent for Logistic Regression

$$J_{\text{reg}}(\boldsymbol{\theta}) = - \sum_{i=1}^n \left[y^{(i)} \log h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) \right] + \lambda \|\boldsymbol{\theta}_{[1:d]}\|_2^2$$

Want $\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$

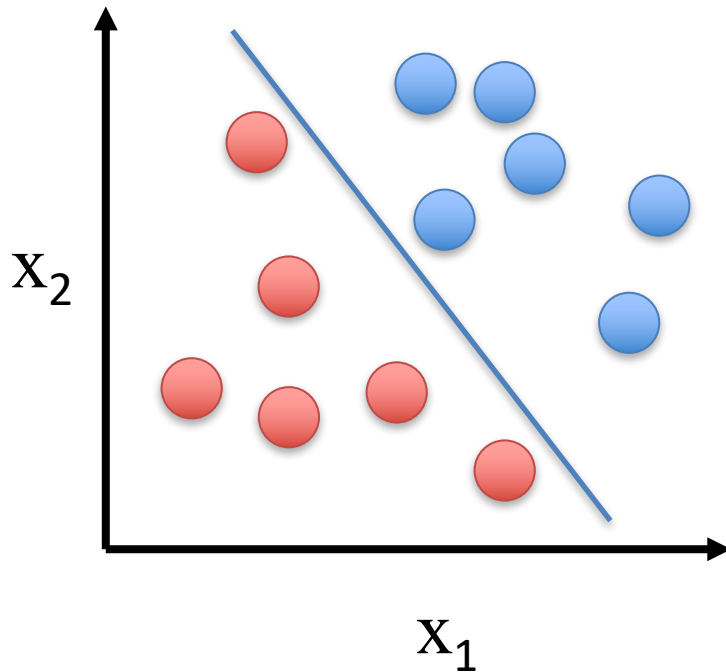
- Initialize $\boldsymbol{\theta}$
- Repeat until convergence (simultaneous update for $j = 0 \dots d$)

$$\theta_0 \leftarrow \theta_0 - \alpha \sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right)$$

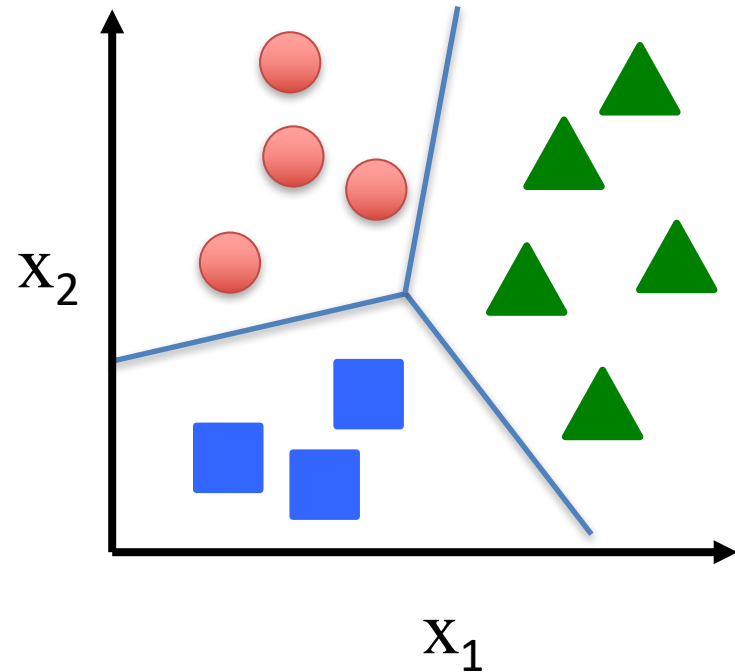
$$\theta_j \leftarrow \theta_j - \alpha \left[\sum_{i=1}^n \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} - \frac{\lambda}{n} \theta_j \right]$$

Multi-Class Classification

Binary classification:

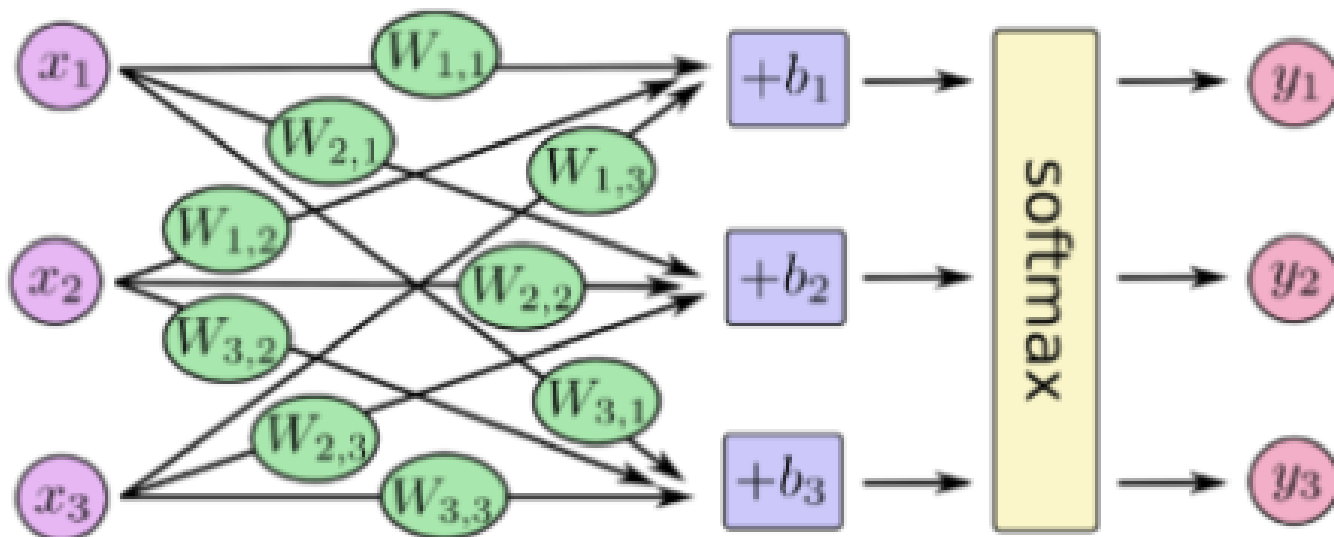


Multi-class classification:



Disease diagnosis: healthy / cold / flu / pneumonia

Object classification: desk / chair / monitor / bookcase



If we write it as an equation, we can get:

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \text{softmax} \left(\begin{bmatrix} W_{1,1}x_1 + W_{1,2}x_1 + W_{1,3}x_1 + b_1 \\ W_{2,1}x_2 + W_{2,2}x_2 + W_{2,3}x_2 + b_2 \\ W_{3,1}x_3 + W_{3,2}x_3 + W_{3,3}x_3 + b_3 \end{bmatrix} \right)$$

Multi-Class Logistic Regression

- For 2 classes:

$$h_{\theta}(x) = \frac{1}{1 + \exp(-\theta^T x)} = \frac{\exp(\theta^T x)}{1 + \exp(\theta^T x)}$$

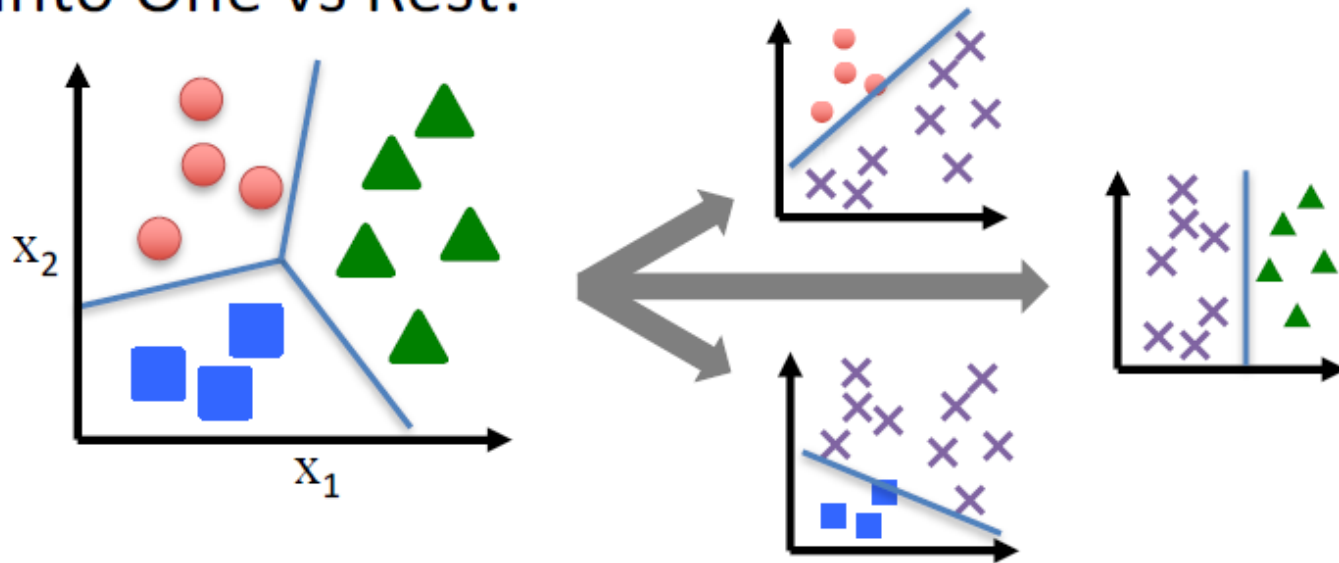
- For C classes $\{1, \dots, C\}$:

$$p(y = c \mid x; \theta_1, \dots, \theta_C) = \frac{\exp(\theta_c^T x)}{\sum_{c=1}^C \exp(\theta_c^T x)}$$

- Called the **softmax** function

Multi-Class Logistic Regression

Split into One vs Rest:



- Train a logistic regression classifier for each class i to predict the probability that $y = i$ with

$$h_c(\mathbf{x}) = \frac{\exp(\boldsymbol{\theta}_c^T \mathbf{x})}{\sum_{c=1}^C \exp(\boldsymbol{\theta}_c^T \mathbf{x})}$$

Implementing Multi-Class Logistic Regression

- Use $h_c(\mathbf{x}) = \frac{\exp(\boldsymbol{\theta}_c^\top \mathbf{x})}{\sum_{c=1}^C \exp(\boldsymbol{\theta}_c^\top \mathbf{x})}$ as the model for class c
- Gradient descent simultaneously updates all parameters for all models
 - Same derivative as before, just with the above $h_c(\mathbf{x})$
- Predict class label as the most probable label

$$\max_c h_c(\mathbf{x})$$